

FR. CONCEICAO RODRIGUES COLLEGE OF ENGINEERING

Department of Computer Engineering

Academic Year 2025-2026

Rubrics for Lab Experiments

Class: B.E. Computer
Engineering

Subject Name: SMA Lab

Semster: VIII

Subject Code: CSDL8023

Practical No.	5
Title:	Develop Content (text, emoticons, image, audio, video) based social media analytics model for business. (e.g. Content Based Analysis: Topic, Issue, Trend, sentiment/opinion analysis, audio, video, image analytics)
Date of Performance:	29-11-2025
Roll No.	9913
Student	Mark Lopes

Evaluation

Performance Indicator	Below Average	Average	Good	Excellent	Marks
On Time Submission (2)	Not submitted(0)	Submitted after deadline (1)	Early or on time submission (2)	—	
Test cases and output (4)	Incorrect output (1)	The expected output is verified only a for few test cases (2)	The expected output is Verified for all test cases but is not presentable (3)	Expected output is obtained for all test cases. Presentable and easy to follow (4)	
Coding efficiency (2)	The code is not structured at all (0)	The code is structured but not efficient (1)	The code is Structured and efficient. (2)	-	
Knowledge(2)	Basic concepts not clear (0)	Understood the basic concepts (1)	Could explain the concept with suitable example (1.5)	Could relate the theory with real world application(2)	
Total					

Signature of the Teacher:

Aim: Develop Content (text, emoticons, image, audio, video) based social media analytics model for business. (e.g. Content Based Analysis: Topic, Issue, Trend, sentiment/opinion analysis, audio, video, image analytics)

Code:

./main.py

```
# =====
# PART 1: IMPORT NECESSARY LIBRARIES
# =====

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from collections import Counter
import warnings
warnings.filterwarnings('ignore')

# Text Processing & NLP
import nltk
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
from nltk.sentiment import SentimentIntensityAnalyzer
import re

# Topic Modeling
from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer
from sklearn.decomposition import LatentDirichletAllocation, NMF

# Image Processing
from PIL import Image
import cv2

# Audio Processing
import librosa
import librosa.display

# Video Processing
from moviepy.editor import VideoFileClip

# Social Media APIs (install as needed)
# pip install tweepy snsrape praw instaloader
# pip install nltk scikit-learn pandas matplotlib seaborn
# pip install pillow opencv-python librosa moviepy

# Download NLTK data
# Ensure 'punkt' is downloaded, as it's critical for tokenization
try:
    nltk.data.find('tokenizers/punkt')
    print("NLTK 'punkt' already downloaded.")
except LookupError:
    print("NLTK 'punkt' resource not found. Downloading...")
nltk.download('punkt', quiet=True) # Force download to ensure punkt_tab is included
print("NLTK 'punkt' download complete.")
```

```

try:
    nltk.data.find('tokenizers/punkt_tab')
    print("NLTK 'punkt_tab' already downloaded.")
except LookupError:
    print("NLTK 'punkt_tab' resource not found. Downloading...")
nltk.download('punkt_tab', quiet=True)
print("NLTK 'punkt_tab' download complete.")

# Check if 'stopwords' is already downloaded
try:
    nltk.data.find('corpora/stopwords')
    print("NLTK 'stopwords' already downloaded.")
except LookupError:
    print("NLTK 'stopwords' resource not found. Downloading...")
    nltk.download('stopwords', quiet=True)
    print("NLTK 'stopwords' download complete.")

# Check if 'vader_lexicon' is already downloaded
try:
    nltk.data.find('sentiment/vader_lexicon')
    print("NLTK 'vader_lexicon' already downloaded.")
except LookupError:
    print("NLTK 'vader_lexicon' resource not found. Downloading...")
    nltk.download('vader_lexicon', quiet=True)
    print("NLTK 'vader_lexicon' download complete.")

# Check if 'averaged_perceptron_tagger' is already downloaded
try:
    nltk.data.find('taggers/averaged_perceptron_tagger')
    print("NLTK 'averaged_perceptron_tagger' already downloaded.")
except LookupError:
    print("NLTK 'averaged_perceptron_tagger' resource not found. Downloading...")
    nltk.download('averaged_perceptron_tagger', quiet=True)
    print("NLTK 'averaged_perceptron_tagger' download complete.")

# =====
# PART 2: DATA COLLECTION FROM REAL SOURCES
# =====

class SocialMediaDataCollector:
    """
    Collect real data from various social media platforms
    """

    def __init__(self):
        self.data = []

    # METHOD 1: Collect from CSV/Excel files
    def load_from_file(self, file_path):
        """
        Load social media data from CSV or Excel file
        Expected columns: text, timestamp, likes, shares, comments
        """
        try:
            if file_path.endswith('.csv'):
                df = pd.read_csv(file_path)
            elif file_path.endswith('.xlsx'):
                df = pd.read_excel(file_path)
            else:

```

```

        print("Unsupported file format. Use CSV or Excel.")
        return None

    print(f"Loaded {len(df)} posts from {file_path}")
    return df
except Exception as e:
    print(f"Error loading file: {e}")
    return None

# METHOD 2: Collect from Twitter/X using snsrape (no API key needed)
def collect_twitter_data(self, query, max_tweets=100):
    """
    Collect tweets using snsrape (no API authentication required)
    Install: pip install snsrape
    """
    try:
        import snsrape.modules.twitter as sntwitter

        tweets_data = []
        for i, tweet in enumerate(sntwitter.TwitterSearchScraper(query).get_items()):
            if i >= max_tweets:
                break

            tweets_data.append({
                'text': tweet.content,
                'date': tweet.date,
                'likes': tweet.likeCount,
                'retweets': tweet.retweetCount,
                'replies': tweet.replyCount,
                'username': tweet.user.username,
                'hashtags': tweet.hashtags if tweet.hashtags else []
            })

        df = pd.DataFrame(tweets_data)
        print(f"Collected {len(df)} tweets")
        return df

    except ImportError:
        print("snsrape not installed. Install with: pip install snsrape")
        return None
    except Exception as e:
        print(f"Error collecting Twitter data: {e}")
        return None

# METHOD 3: Collect from Reddit
def collect_reddit_data(self, subreddit_name, limit=100):
    """
    Collect posts from Reddit
    Install: pip install praw
    You'll need Reddit API credentials from: https://www.reddit.com/prefs/apps
    """
    try:
        import praw

        # You need to create an app at https://www.reddit.com/prefs/apps
        reddit = praw.Reddit(
            client_id='YOUR_CLIENT_ID',
            client_secret='YOUR_CLIENT_SECRET',
            user_agent='SocialMediaAnalytics/1.0'

```

```

    )

    subreddit = reddit.subreddit(subreddit_name)
    posts_data = []

    for post in subreddit.hot(limit=limit):
        posts_data.append({
            'text': post.title + " " + post.selftext,
            'date': pd.to_datetime(post.created_utc, unit='s'),
            'score': post.score,
            'comments': post.num_comments,
            'author': str(post.author),
            'url': post.url
        })

    df = pd.DataFrame(posts_data)
    print(f"Collected {len(df)} Reddit posts")
    return df

except ImportError:
    print("praw not installed. Install with: pip install praw")
    return None
except Exception as e:
    print(f"Error collecting Reddit data: {e}")
    return None

# METHOD 4: Load sample dataset for testing
def load_sample_data(self):
    """
    Create sample data for testing when APIs are unavailable
    """
    sample_data = {
        'text': [
            "Absolutely love this new product! Best purchase ever! #Amazing",
            "Terrible customer service. Very disappointed with my experience.",
            "The quality is okay, nothing special but gets the job done.",
            "Wow! This exceeded all my expectations. Highly recommend!",
            "Not worth the price. Expected much better quality.",
            "Great value for money. Happy with my purchase!",
            "The delivery was late and the product arrived damaged.",
            "Excellent features and easy to use. Five stars!",
            "Could be better. Some features don't work as advertised.",
            "Amazing quality! Will definitely buy again. #BestProduct"
        ],
        'date': pd.date_range(start='2024-01-01', periods=10, freq='D'),
        'likes': [245, 34, 89, 312, 45, 198, 23, 267, 78, 289],
        'shares': [45, 12, 23, 67, 8, 34, 5, 56, 15, 61],
        'comments': [23, 8, 12, 45, 6, 28, 4, 38, 11, 42]
    }

    df = pd.DataFrame(sample_data)
    print(f"Loaded {len(df)} sample posts")
    return df

```

```

# =====
# PART 3: DATA PREPROCESSING
# =====

```

```

class DataPreprocessor:

```

```
"""
Clean and preprocess text data
"""
```

```
def __init__(self):
    self.stop_words = set(stopwords.words('english'))
```

```
def clean_text(self, text):
    """
    Clean text by removing special characters, URLs, mentions
    """
    if pd.isna(text):
        return ""

    # Convert to lowercase
    text = str(text).lower()

    # Remove URLs
    text = re.sub(r'http\S+|www\S+|https\S+', '', text, flags=re.MULTILINE)

    # Remove mentions and hashtags (but keep the text)
    text = re.sub(r'@\w+', '', text)
    text = re.sub(r'#', '', text)

    # Remove special characters and digits
    text = re.sub(r'[^a-zA-Z\s]', '', text)

    # Remove extra whitespace
    text = ' '.join(text.split())

    return text
```

```
def tokenize_text(self, text):
    """
    Tokenize text and remove stopwords
    """
    tokens = word_tokenize(text)
    tokens = [word for word in tokens if word not in self.stop_words and len(word) > 2]
    return tokens
```

```
def preprocess_dataframe(self, df, text_column='text'):
    """
    Preprocess entire dataframe
    """
    df['cleaned_text'] = df[text_column].apply(self.clean_text)
    df['tokens'] = df['cleaned_text'].apply(self.tokenize_text)
    df['token_count'] = df['tokens'].apply(len)

    return df
```

```
# =====
# PART 4: SENTIMENT ANALYSIS USING NLTK
# =====
```

```
class SentimentAnalyzer:
    """
    Perform sentiment analysis using NLTK's VADER
    """
```

```

def __init__(self):
    self.sia = SentimentIntensityAnalyzer()

def analyze_sentiment(self, text):
    """
    Analyze sentiment of a single text
    Returns: dict with pos, neg, neu, compound scores
    """
    if pd.isna(text) or text == "":
        return {'neg': 0, 'neu': 1, 'pos': 0, 'compound': 0}

    scores = self.sia.polarity_scores(text)
    return scores

def classify_sentiment(self, compound_score):
    """
    Classify sentiment based on compound score
    """
    if compound_score >= 0.05:
        return 'Positive'
    elif compound_score <= -0.05:
        return 'Negative'
    else:
        return 'Neutral'

def analyze_dataframe(self, df, text_column='text'):
    """
    Analyze sentiment for entire dataframe
    """
    # Get sentiment scores
    df['sentiment_scores'] = df[text_column].apply(self.analyze_sentiment)

    # Extract individual scores
    df['positive_score'] = df['sentiment_scores'].apply(lambda x: x['pos'])
    df['negative_score'] = df['sentiment_scores'].apply(lambda x: x['neg'])
    df['neutral_score'] = df['sentiment_scores'].apply(lambda x: x['neu'])
    df['compound_score'] = df['sentiment_scores'].apply(lambda x: x['compound'])

    # Classify sentiment
    df['sentiment'] = df['compound_score'].apply(self.classify_sentiment)

    return df

def visualize_sentiment(self, df):
    """
    Visualize sentiment distribution
    """
    fig, axes = plt.subplots(1, 2, figsize=(15, 5))

    # Pie chart
    sentiment_counts = df['sentiment'].value_counts()
    colors = ['#10b981', '#3b82f6', '#ef4444']
    axes[0].pie(sentiment_counts, labels=sentiment_counts.index, autopct='%1.1f%%',
                colors=colors, startangle=90)
    axes[0].set_title('Sentiment Distribution', fontsize=14, fontweight='bold')

    # Bar chart
    sentiment_counts.plot(kind='bar', ax=axes[1], color=colors)
    axes[1].set_title('Sentiment Counts', fontsize=14, fontweight='bold')

```

```

axes[1].set_xlabel('Sentiment')
axes[1].set_ylabel('Count')
axes[1].tick_params(axis='x', rotation=0)

```

```

plt.tight_layout()
plt.show()

```

```

# Print summary
print("\n=== SENTIMENT ANALYSIS SUMMARY ===")
print(f"Total Posts: {len(df)}")
print(f"\nSentiment Distribution:")
for sentiment, count in sentiment_counts.items():
    percentage = (count / len(df)) * 100
    print(f" {sentiment}: {count} ({percentage:.1f}%)")
print(f"\nAverage Compound Score: {df['compound_score'].mean():.3f}")

```

```

# =====
# PART 5: TOPIC MODELING USING LDA
# =====

```

```

class TopicModeler:
    """
    Perform topic modeling using Latent Dirichlet Allocation
    """

    def __init__(self, n_topics=5, max_features=1000):
        self.n_topics = n_topics
        self.max_features = max_features
        self.vectorizer = None
        self.lda_model = None

    def fit_lda(self, texts):
        """
        Fit LDA model on text data
        """
        # Create document-term matrix
        self.vectorizer = CountVectorizer(
            max_features=self.max_features,
            max_df=0.95,
            min_df=2,
            stop_words='english'
        )

        doc_term_matrix = self.vectorizer.fit_transform(texts)

        # Fit LDA model
        self.lda_model = LatentDirichletAllocation(
            n_components=self.n_topics,
            random_state=42,
            max_iter=20
        )

        self.lda_model.fit(doc_term_matrix)

        return doc_term_matrix

    def get_topics(self, n_words=10):
        """
        Extract top words for each topic

```



```

"""
feature_names = self.vectorizer.get_feature_names_out()
topics = []

for topic_idx, topic in enumerate(self.lda_model.components_):
    top_words_idx = topic.argsort()[-n_words:][::-1]
    top_words = [feature_names[i] for i in top_words_idx]
    topics.append({
        'topic_idx': topic_idx,
        'words': top_words,
        'weights': topic[top_words_idx]
    })

return topics

def visualize_topics(self, topics):
    """
    Visualize topics
    """
    print("\n=== TOPIC MODELING RESULTS (LDA) ===")
    print(f"Number of Topics: {self.n_topics}\n")

    for topic in topics:
        print(f"Topic {topic['topic_idx']}:")
        print(f"  Top words: {' '.join(topic['words'][:8])}")
        print()

    # Create visualization
    fig, axes = plt.subplots(self.n_topics, 1, figsize=(12, self.n_topics * 2))

    if self.n_topics == 1:
        axes = [axes]

    for idx, topic in enumerate(topics):
        words = topic['words'][:10]
        weights = topic['weights'][:10]

        axes[idx].barh(words, weights, color='#3b82f6')
        axes[idx].set_title(f'Topic {idx}', fontweight='bold')
        axes[idx].set_xlabel('Weight')
        axes[idx].invert_yaxis()

    plt.tight_layout()
    plt.show()

# =====
# PART 6: AUDIO PROCESSING USING LIBROSA
# =====

class AudioAnalyzer:
    """
    Analyze audio files using LibROSA
    """

    def __init__(self):
        self.sample_rate = 22050

    def load_audio(self, audio_path):
        """

```

```

Load audio file
"""
try:
    audio, sr = librosa.load(audio_path, sr=self.sample_rate)
    return audio, sr
except Exception as e:
    print(f"Error loading audio: {e}")
    return None, None

def extract_features(self, audio, sr):
    """
    Extract audio features
    """
    features = {}

    # Tempo and beat
    tempo, _ = librosa.beat.beat_track(y=audio, sr=sr)
    features['tempo'] = tempo

    # Zero crossing rate (indicates percussiveness)
    zcr = librosa.feature.zero_crossing_rate(audio)
    features['zero_crossing_rate'] = float(np.mean(zcr))

    # Spectral features
    spectral_centroids = librosa.feature.spectral_centroid(y=audio, sr=sr)
    features['spectral_centroid'] = float(np.mean(spectral_centroids))

    # MFCCs (timbre)
    mfccs = librosa.feature.mfcc(y=audio, sr=sr, n_mfcc=13)
    features['mfcc_mean'] = float(np.mean(mfccs))
    features['mfcc_std'] = float(np.std(mfccs))

    # Chroma features (pitch)
    chroma = librosa.feature.chroma_stft(y=audio, sr=sr)
    features['chroma_mean'] = float(np.mean(chroma))

    # RMS Energy
    rms = librosa.feature.rms(y=audio)
    features['rms_energy'] = float(np.mean(rms))

    return features

def visualize_audio(self, audio, sr, audio_name="Audio"):
    """
    Visualize audio waveform and spectrogram
    """
    fig, axes = plt.subplots(2, 1, figsize=(14, 8))

    # Waveform
    librosa.display.waveshow(audio, sr=sr, ax=axes[0])
    axes[0].set_title(f'{audio_name} - Waveform', fontsize=14, fontweight='bold')
    axes[0].set_xlabel('Time (s)')
    axes[0].set_ylabel('Amplitude')

    # Spectrogram
    D = librosa.amplitude_to_db(np.abs(librosa.stft(audio)), ref=np.max)
    img = librosa.display.specshow(D, sr=sr, x_axis='time', y_axis='hz', ax=axes[1])
    axes[1].set_title(f'{audio_name} - Spectrogram', fontsize=14, fontweight='bold')
    fig.colorbar(img, ax=axes[1], format='%+2.0f dB')

```

```

plt.tight_layout()
plt.show()

# =====
# PART 7: COMPLETE ANALYSIS PIPELINE
# =====

def run_complete_analysis():
    """
    Run complete social media analytics pipeline
    """
    print("=" * 70)
    print("SOCIAL MEDIA ANALYTICS - REAL DATA IMPLEMENTATION")
    print("=" * 70)

    # Step 1: Collect Data
    print("\n[STEP 1] DATA COLLECTION")
    print("-" * 70)
    collector = SocialMediaDataCollector()

    # Choose your data source:)
    # Option 1: Load from file (if you have CSV/Excel)
    # df = collector.load_from_file('your_social_media_data.csv')

    # Option 2: Collect from Twitter
    # df = collector.collect_twitter_data('#YourHashtag', max_tweets=100)

    # Option 3: Collect from Reddit
    # df = collector.collect_reddit_data('technology', limit=100)

    # Option 4: Use sample data (for testing)
    df = collector.load_sample_data()

    if df is None or len(df) == 0:
        print("No data collected. Exiting...")
        return

    print(f"✓ Successfully loaded {len(df)} posts\n")

    # Step 2: Preprocess Data
    print("[STEP 2] DATA PREPROCESSING")
    print("-" * 70)
    preprocessor = DataPreprocessor()
    df = preprocessor.preprocess_dataframe(df)
    print(f"✓ Data preprocessing complete")
    print(f"✓ Average tokens per post: {df['token_count'].mean():.1f}\n")

    # Step 3: Sentiment Analysis
    print("[STEP 3] SENTIMENT ANALYSIS (NLTK VADER)")
    print("-" * 70)
    sentiment_analyzer = SentimentAnalyzer()
    df = sentiment_analyzer.analyze_dataframe(df)
    sentiment_analyzer.visualize_sentiment(df)

    # Step 4: Topic Modeling
    print("\n[STEP 4] TOPIC MODELING (LDA)")
    print("-" * 70)
    topic_modeler = TopicModeler(n_topics=3, max_features=500)

```

```

doc_term_matrix = topic_modeler.fit_lda(df['cleaned_text'])
topics = topic_modeler.get_topics(n_words=10)
topic_modeler.visualize_topics(topics)

# Step 5: Create Search Prompt
print("\n[STEP 5] TEXT INSIGHT SEARCH")
print("-" * 70)
search_keyword = input("Enter keyword to search insights: ").lower()

matching_posts = df[df['cleaned_text'].str.contains(search_keyword, na=False)]

if len(matching_posts) > 0:
    print(f"\n✓ Found {len(matching_posts)} posts containing '{search_keyword}'")
    print(f"\nSentiment breakdown:")
    print(matching_posts['sentiment'].value_counts())
    print(f"\nAverage engagement:")
    if 'likes' in matching_posts.columns:
        print(f"    Likes: {matching_posts['likes'].mean():.1f}")
    if 'shares' in matching_posts.columns:
        print(f"    Shares: {matching_posts['shares'].mean():.1f}")
else:
    print(f"x No posts found containing '{search_keyword}'")

# Step 6: Export Results
print("\n[STEP 6] EXPORT RESULTS")
print("-" * 70)
output_file = 'social_media_analysis_results.csv'
df.to_csv(output_file, index=False)
print(f"✓ Results exported to: {output_file}")

print("\n" + "=" * 70)
print("ANALYSIS COMPLETE!")
print("=" * 70)

return df

```

```

# =====
# PART 8: ADDITIONAL UTILITIES
# =====

```

```

def analyze_audio_file(audio_path):
    """
    Standalone function to analyze audio file
    """
    print(f"\nAnalyzing audio file: {audio_path}")
    analyzer = AudioAnalyzer()
    audio, sr = analyzer.load_audio(audio_path)

    if audio is not None:
        features = analyzer.extract_features(audio, sr)
        print("\nAudio Features:")
        for key, value in features.items():
            print(f"    {key}: {value:.2f}")

        analyzer.visualize_audio(audio, sr, audio_path)
        return features
    return None

def create_engagement_report(df):

```

```

"""
Create engagement metrics report
"""

print("\n=== ENGAGEMENT REPORT ===")

if 'likes' in df.columns:
    print(f"\nLikes:")
    print(f"  Total: {df['likes'].sum()}")
    print(f"  Average: {df['likes'].mean():.1f}")
    print(f"  Max: {df['likes'].max()}")

if 'shares' in df.columns:
    print(f"\nShares:")
    print(f"  Total: {df['shares'].sum()}")
    print(f"  Average: {df['shares'].mean():.1f}")
    print(f"  Max: {df['shares'].max()}")

if 'comments' in df.columns:
    print(f"\nComments:")
    print(f"  Total: {df['comments'].sum()}")
    print(f"  Average: {df['comments'].mean():.1f}")
    print(f"  Max: {df['comments'].max()}")

# Top performing posts
print("\n=== TOP 5 POSTS BY ENGAGEMENT ===")
if 'likes' in df.columns:
    top_posts = df.nlargest(5, 'likes')[['text', 'likes', 'sentiment']]
    for idx, (_, row) in enumerate(top_posts.iterrows(), 1):
        print(f"\n{idx}. {row['text'][:100]}...")
        print(f"  Likes: {row['likes']} | Sentiment: {row['sentiment']}")

# =====
# MAIN EXECUTION
# =====

if __name__ == "__main__":
    # Run complete analysis
    df = run_complete_analysis()

    # Optional: Create engagement report
    if df is not None:
        create_engagement_report(df)

    # Optional: Analyze audio file (if you have audio files)
    # analyze_audio_file('path_to_your_audio.mp3')

    print("\n✓ All analyses completed successfully!")

```

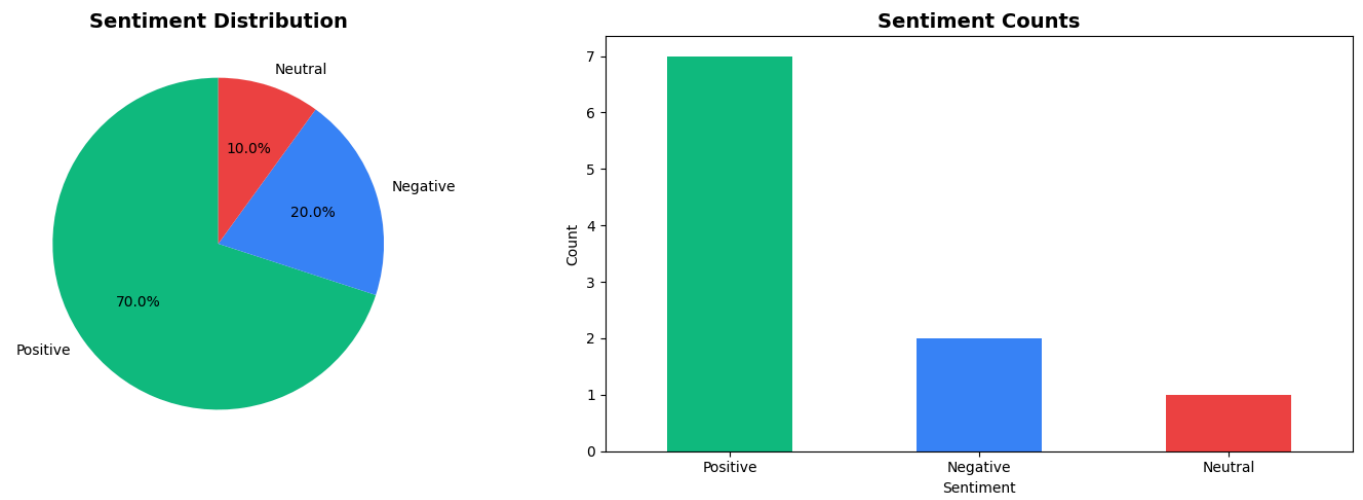
Output:

```
NLTK 'punkt' already downloaded.
NLTK 'punkt' download complete.
NLTK 'punkt_tab' resource not found. Downloading...
NLTK 'punkt_tab' download complete.
NLTK 'stopwords' already downloaded.
NLTK 'vader_lexicon' resource not found. Downloading...
NLTK 'vader_lexicon' download complete.
NLTK 'averaged_perceptron_tagger' already downloaded.
=====
SOCIAL MEDIA ANALYTICS - REAL DATA IMPLEMENTATION
=====

[STEP 1] DATA COLLECTION
-----
Loaded 10 sample posts
✓ Successfully loaded 10 posts

[STEP 2] DATA PREPROCESSING
-----
✓ Data preprocessing complete
✓ Average tokens per post: 5.8

[STEP 3] SENTIMENT ANALYSIS (NLTK VADER)
```



```
=== SENTIMENT ANALYSIS SUMMARY ===
Total Posts: 10

Sentiment Distribution:
  Positive: 7 (70.0%)
  Negative: 2 (20.0%)
  Neutral: 1 (10.0%)

Average Compound Score: 0.363

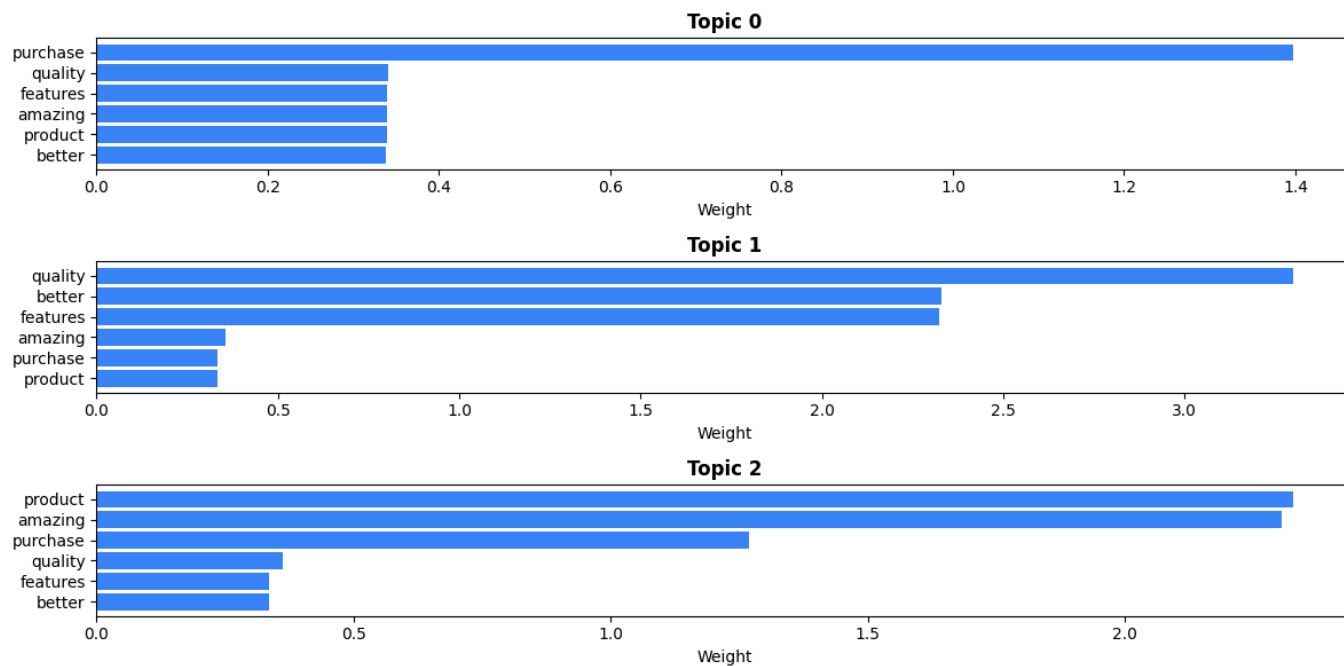
[STEP 4] TOPIC MODELING (LDA)
-----

=== TOPIC MODELING RESULTS (LDA) ===
Number of Topics: 3

Topic 0:
  Top words: purchase, quality, features, amazing, product, better

Topic 1:
  Top words: quality, better, features, amazing, purchase, product

Topic 2:
  Top words: product, amazing, purchase, quality, features, better
```



```
[STEP 5] TEXT INSIGHT SEARCH
-----
Enter keyword to search insights: good
X No posts found containing 'good'

[STEP 6] EXPORT RESULTS
-----
✓ Results exported to: social_media_analysis_results.csv

=====
ANALYSIS COMPLETE!
=====

=== ENGAGEMENT REPORT ===

Likes:
  Total: 1580
  Average: 158.0
  Max: 312

Shares:
  Total: 326
  Average: 32.6
  Max: 67

Comments:
  Total: 217
  Average: 21.7
  Max: 45

=== TOP 5 POSTS BY ENGAGEMENT ===

1. Wow! This exceeded all my expectations. Highly recommend!...
   Likes: 312 | Sentiment: Positive

2. Amazing quality! Will definitely buy again. #BestProduct...
   Likes: 289 | Sentiment: Positive

3. Excellent features and easy to use. Five stars!...
   Likes: 267 | Sentiment: Positive

4. Absolutely love this new product! Best purchase ever! #Amazing...
   Likes: 245 | Sentiment: Positive

5. Great value for money. Happy with my purchase!...
   Likes: 198 | Sentiment: Positive

✓ All analyses completed successfully!
```